



PDF Download
3759458.pdf
02 April 2026
Total Citations: 0
Total Downloads: 789

 Latest updates: <https://dl.acm.org/doi/10.1145/3759458>

RESEARCH-ARTICLE

HMSA: High-Performance Heterogeneous Mixed-Precision CNN Systolic Array Accelerator on FPGA

YONGXIANG CAO, Beihang University, Beijing, China

HONGXU JIANG, Beihang University, Beijing, China

HUIYONG LI, Beihang University, Beijing, China

YU TANG, Beihang University, Beijing, China

DONGCHENG SHI, Beihang University, Beijing, China

GUOCHENG ZHAO, Beihang University, Beijing, China

Open Access Support provided by:

Beihang University

Published: 26 September 2025

Online AM: 07 August 2025

Accepted: 29 July 2025

Revised: 22 July 2025

Received: 22 July 2025

[Citation in BibTeX format](#)

HMSA: High-Performance Heterogeneous Mixed-Precision CNN Systolic Array Accelerator on FPGA

YONGXIANG CAO, School of Computer Science and Engineering, Beihang University, Beijing, China

HONGXU JIANG*, School of Computer Science and Engineering, Beihang University, Beijing, China

HUIYONG LI, School of Artificial Intelligence, Beihang University, Beijing, China

YU TANG, School of Computer Science and Engineering, Beihang University, Beijing, China

DONGCHENG SHI, School of Computer Science and Engineering, Beihang University, Beijing, China

GUOCHENG ZHAO, School of Computer Science and Engineering, Beihang University, Beijing, China

In power-constrained and real-time-demanding embedded scenarios, Field-Programmable Gate Arrays (FPGAs) emerge as ideal options for accelerating neural network inference, owing to the reconfigurability, high reliability, and flexibility of FPGAs. Mixed precision quantization technology significantly reduces computational complexity and bandwidth requirements while preserving model accuracy. However, existing FPGA accelerators fail to fully leverage the parallel advantages of mixed precision, which leads to the actual inference speedup being markedly lower than the theoretical prediction. Reviewing existing methods, we found three main drawbacks in enhancing practical computational performance. Firstly, FPGAs primarily rely on DSP slices to achieve high-performance parallel multiplication and accumulation (MAC) in neural network inference. However, the current DSP PE design and data packing methods are not compatible with mixed-precision models. Secondly, the remaining logic resources are not fully utilized to accelerate computations. Thirdly, the mixed-precision quantization bit-width selection method without hardware-guided guidance leads to additional model accuracy loss. To address these challenges, we propose a high-performance heterogeneous mixed-precision systolic array (SA) accelerator, HMSA. It aims to leverage mixed-precision quantization fully, enhancing the practical inference efficiency of neural networks on embedded FPGAs. We propose an optimized DSP data packing method guided by a resource-performance cost model, which enhances the parallel computing performance of accelerators. We propose a heterogeneous convolutional acceleration architecture based on SA architecture with high scalability, enabling efficient utilization of FPGA's heterogeneous computing resources. In terms of the algorithm, we propose an optimization method for bit-width selection based on FPGA hardware architecture, aiming to avoid accuracy degradation without improving inference speed. Experiments confirm that HMSA on the Xilinx XC7VX690T FPGA reaches a peak throughput of 6.385 TOP/s at W1A8 precision. When inferring mixed precision neural networks, HMSA achieves 3.53 $\times$, 5.46 $\times$, and 1.58 $\times$ improvements in actual throughput/DSP compared to state-of-the-art MPA, MSD, and MP-OPU.

This work is supported by the China Higher Education Institution Industry-University-Research Innovation Fund (No. 2024HY001).

Authors' Contact Information: Yongxiang Cao, School of Computer Science and Engineering, Beihang University, Beijing, China; e-mail: caoyongxiang@buaa.edu.cn; Hongxu Jiang, School of Computer Science and Engineering, Beihang University, Beijing, China; e-mail: jianghx@buaa.edu.cn; Huiyong Li, School of Artificial Intelligence, Beihang University, Beijing, China; e-mail: lihuiyong@buaa.edu.cn; Yu Tang, School of Computer Science and Engineering, Beihang University, Beijing, China; e-mail: tangy@buaa.edu.cn; Dongcheng Shi, School of Computer Science and Engineering, Beihang University, Beijing, China; e-mail: shidongcheng@buaa.edu.cn; Guocheng Zhao, School of Computer Science and Engineering, Beihang University, Beijing, China; e-mail: zhaoguocheng@buaa.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1539-9087/2025/09-ART107

<https://doi.org/10.1145/3759458>

Compared with the state-of-the-art MBFQuant and Edge-MPQ mixed quantization algorithms, the proposed optimized bit-width selection method effectively reduces the model accuracy loss.

CCS Concepts: • **Hardware** → **Hardware accelerators**; • **Computer systems organization** → **Neural networks**; **Systolic arrays**; **Heterogeneous (hybrid) systems**;

Additional Key Words and Phrases: Mixed-precision, systolic array, FPGA, hardware acceleration

ACM Reference Format:

Yongxiang Cao, Hongxu Jiang, Huiyong Li, Yu Tang, Dongcheng Shi, and Guocheng Zhao. 2025. HMSA: High-Performance Heterogeneous Mixed-Precision CNN Systolic Array Accelerator on FPGA. *ACM Trans. Embedd. Comput. Syst.* 24, 5s, Article 107 (September 2025), 27 pages. <https://doi.org/10.1145/3759458>

1 Introduction

Embedded devices are characterized by strict constraints in power consumption, computational resources, hardware storage capacity, and transmission bandwidth [41]. To address these challenges, **mixed-precision quantization (MPQ)** has emerged as a key technique for neural network compression [28]. It effectively reduces the computational complexity and storage requirements of neural network models with minimal accuracy loss. However, AI-specific accelerators lack sufficient support for mixed-precision neural networks, such as TPUs and NPU [27]. For example, when mapping a 2-bit quantized network onto these architectures, it often needs to be converted to either 4-bit or 8-bit formats. This conversion can lead to either accuracy degradation or increased latency. In contrast to other hardware, FPGAs exhibit rapid adaptability to algorithmic advancements [13]. They effectively exploit mixed-precision capabilities, while featuring low power consumption, strong scalability, and high stability [6]. These advantages make FPGAs a favored option for deploying **deep neural networks (DNNs)** on embedded platforms [29]. As a result, improving the performance of mixed-precision accelerators on FPGAs for optimal execution of mixed-precision quantized and compressed neural networks has emerged as a key research focus in this field.

Multiplication and addition are the primary computational costs in DNN inference. FPGA incorporates various programmable logic resources, among which **Digital Signal Processor (DSP)** blocks and **Lookup Tables (LUTs)** are two key components. DSP blocks are specialized hardware units in FPGAs designed for efficiently performing digital signal processing operations. They are typically optimized to accelerate multiplication, addition, and other arithmetic computations, thereby significantly improving computational efficiency. LUTs serve as the fundamental building blocks for implementing combinational logic in FPGAs. By configuring their contents, any Boolean logic function can be realized. In accelerator hardware design, efficiently utilizing DSP resources in FPGAs is critical for implementing high-performance parallel multiplications and additions [3, 26, 39, 40]. In the domain of hardware design, existing research has neglected two critical factors. First, the research has failed to fully exploit the input bit-width of DSP slices. Moreover, the relationship between the parallelism of multiply-accumulate operations within DSPs and the overall parallel processing capability of the accelerator has not been adequately addressed. The accelerator's performance has the potential for further optimization. Second, some studies propose designing heterogeneous acceleration engines by leveraging DSP and LUT computational resources to enhance accelerator throughput [12, 18, 40]. However, these engines introduce significant logic resource overhead when supporting high-bit-width computations, thereby constraining performance improvements. In the field of MPQ, current research overlooks the DSP structural characteristics of FPGAs during the bit-width selection process [36, 42]. This leads to greater accuracy loss in the quantized model under the same computational performance.

To address the aforementioned challenges, we propose a high-performance heterogeneous mixed-precision DNN accelerator based on SA architecture, HMSA. This design fully exploits on-chip resources (e.g., LUTs, DSPs, and BRAMs) for computation, achieving higher computing performance than existing studies under the same hardware resource constraints. We designed a dynamically configurable heterogeneous hardware architecture that includes D-MAC and L-MAC. This architecture is directed by the cost model to guide the selection of the hardware design architecture type with the highest computational power based on quantized models, and it allows for the automatic generation of hardware accelerators using preset templates. Based on the above acceleration engines, we designed a high-performance heterogeneous accelerator using the SA architecture. In the process of bit-width selection for the MPQ algorithm, we propose a hardware-guided mixed-precision two-stage quantization method. HMSP not only reduces the search space for mixed-precision bit-widths but also avoids generating solutions that degrade the model's accuracy without improving computational performance. The main contributions of this article are as follows:

- We propose a DSP packaging optimization method based on a resource-performance cost model. This method can calculate the parallel design of DSPs in PE with the highest computing power according to the computational load of different bit-width combinations in a mixed-precision DNN.
- We present a high-performance FPGA accelerator architecture with a heterogeneous SA that combines DSP and LUT resources. SA is utilized to alleviate the data sharing and memory access pressure of PEs, reducing the consumption of transmission resources.
- We propose an adaptive architecture selection method based on the precision distribution characteristics of neural network models. This method optimizes the data path design of the DSP computing engine and the architecture type selection of the LUT computing engine by quantitatively analyzing the computational workload proportions of different bit-width combinations in mixed-precision quantized models.
- We propose a hardware-guided mixed-precision two-stage quantization method based on the search space pruning algorithm that leverages the characteristics of FPGAs' DSP architecture, HMSP. This algorithm reduces the bit-width search time for MPQ. Moreover, after the quantization algorithm guides the selection and configuration of the hardware architecture, it can adaptively adjust the model's bit-width combinations according to the hardware architecture characteristics, thereby recovering the model accuracy.
- We implemented a reconfigurable and adaptive heterogeneous mixed-precision SA accelerator on the Xilinx XC7VX690T FPGA, which significantly enhances the actual computational throughput of model inference within limited resources, HMSA. The HMSA accelerator achieves a peak throughput of 6.385 TOP/s under W1A8 precision configuration. When executing mixed-precision neural networks, the proposed design demonstrates enhanced peak computational density with improvements of 4.86 $\times$, 4.26 $\times$, and 2.59 $\times$ compared to state-of-the-art approaches, including MPA, MSD, and MP-OPU. Corresponding practical inference speed measurements reveal 3.53 $\times$, 5.46 $\times$, and 1.58 $\times$ actual throughput/DSP performance gains over these baseline methods.

2 Background and Motivation

In the context of increasing research on high-performance accelerators for embedded neural network applications, such as object detection and tracking [20], general-purpose architectures like CPUs, GPUs, and TPUs face significant limitations in real-time scenarios. The need to manage numerous threads and execute complex instructions introduces considerable latency, while high power consumption and form factor constraints make them less suitable for resource-limited embedded systems. In contrast, FPGA-based accelerators offer key advantages: flexible reconfigurability

for diverse tasks, low-latency real-time processing through hardware pipelining, and superior energy efficiency. These features make them ideal for fixed-model real-time task handling [1, 22]. Consequently, designing neural network accelerators with enhanced computational capabilities under the constraints of limited design resources holds significant importance [6]. MPQ has emerged as a particularly effective strategy in this context. Nevertheless, a significant challenge remains in the practical deployment of mixed-precision quantized neural networks. There is a significant difference between theoretical reasoning speed and actual reasoning speed [37]. The inherent benefits of MPQ techniques remain underexploited. Therefore, considerable research has been directed towards designing mixed-precision neural network accelerators with high performance on embedded FPGA platforms. In this context, we initiated a rigorous and comprehensive inquiry.

2.1 High-performance Mixed Precision PE Based on DSP on FPGA

2.1.1 Unexplored Optimization Potential in Dynamic Data Packing for DSP in FPGA. When FPGAs are employed to accelerate DNN inference, the optimization of DSP slice resources is important. Different vendors' DSP resources exhibit significant differences in computational characteristics. For instance, Intel's FPGA DSPs feature the capability of operation partitioning, enabling the decomposition of a 27×27 multiplication into two 18×19 multiplications [14]. In contrast, Xilinx's DSP48E1 supports 25×18 multiplications [2]. Building on this, ED proposed a fine-grained data packing method that leverages Intel's DSP characteristics to support four 8×8 and eight 4×4 multiplications within a single DSP [3]. Similarly, research on Xilinx has adopted a similar data packing method, allowing a single DSP resource to perform two 8×8 and four 4×4 multiplications simultaneously [40]. Li et al. extended this data packing method to 1–16 bits [19], proposing a flexible DSP packing method that supports convolution with arbitrary bit widths. However, this method fails to optimize the packing of data with different bit widths, resulting in underutilized data parallelism [19]. To further enhance the computational power based on DSP resources, some researchers have adopted approximate computing to replace exact computing, thereby achieving higher data parallelism [11, 34]. For example, DSP-Packing introduced the "Overpacking" technique, which achieves higher density packing by reducing guard bits and proposed an MSB recovery technique to correct computational errors [34]. Ghavami et al. significantly reduced resource consumption through truncation and compensation optimization [11], but approximate computing introduces precision loss, resulting in reduced model accuracy. Notably, when the bit width of weight data is low (e.g., 2 bits or 1 bit), model accuracy is significantly affected [11]. In embedded applications, limited training data due to collection difficulties results in poor model generalization and amplifies the impact of approximate computing errors on model accuracy compared to standard dataset training [11]. Therefore, efficiently utilizing DSP resources to achieve exact computing remains a crucial research direction for deploying neural network accelerators on embedded FPGAs [11]. Liu et al. proposed a flexible FPGA accelerator that leverages the native structure of DSPs to reduce soft logic overhead [21]. MP-OPU dynamically adjusts its computational mode to accommodate diverse quantization strategies and optimizes the memory architecture and access methods, thereby enhancing the computational power (throughput per DSP) released by a single DSP [39]. However, to ensure the accuracy of computational results, this approach necessitates the use of substantial additional logic for result correction. In contrast, MPA employs bit concatenation and sign compensation techniques to pack multiple signed operands (e.g., weights in neural networks) within a single clock cycle, thereby reducing the logical resource cost required for compensating sign computation errors [26]. Despite significant progress in hardware design, existing studies still have limitations. First, although the accelerator's hardware architecture can dynamically adjust the MP-MAC unit's computational mode through software configuration to meet the precision requirements of different layers, this design fails to fully consider the proportion of different

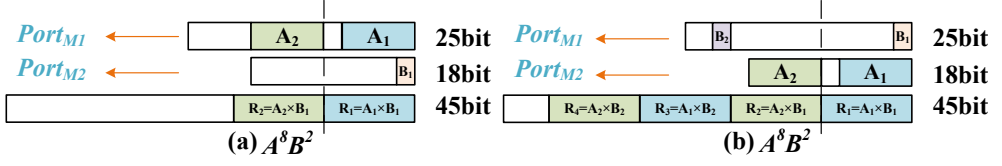


Fig. 1. Swapping the ports for data packing enhances the bit-width utilization of the input ports.

precision combinations in various models under different MPQ strategies [39]. Second, supporting more bit widths, while providing higher flexibility, also increases resource consumption. In the same design resources, supporting multiple mixed precisions in DSP-based PE will lead to lower parallelism between PEs, and even limit the overall computational parallelism [21]. However, existing research has not fully exploited the bit width of the DSP. We take Xilinx's DSP48 as an example. As shown in the Figure 1. $Port_{M1}$ and $Port_{M2}$ are the input ports of DSP. When two 8-bit data, $Data_{A1}$ and $Data_{A2}$, are packed at the $Port_{M1}$ of the DSP, only a 2-bit data $Data_{B1}$ can be input at the $Port_{M2}$. In this case, one DSP can complete two multiplications. However, if the input ports are swapped and two 8-bit data, $Data_{A1}$ and $Data_{A2}$, are packed at the $Port_{M2}$, then two data, $Data_{B1}$ and $Data_{B2}$, can be packed at the $Port_{M1}$. Consequently, a single DSP can execute four multiplication operations concurrently and yield four corresponding results.

Embedded tasks usually have the characteristics of fixed functions and continuous operation, and the reconfigurability of FPGAs enables the accelerator's hardware structure to be rewritten in just a few hours, thereby running the same task on the embedded side for a long time [21]. Many current studies are committed to supporting mixed-precision computing with unchanged hardware but reconfigurable software, fixing the data path hardware within the computing unit and configuring it through software. The advantage of this approach is software configurability, but the downside is that it increases PE design resources and reduces the computational power of the accelerator. In resource-constrained embedded environments, this design may pose challenges to deploying higher computational power accelerators [21, 26, 39].

2.1.2 PE Internal Parallelism Vs. Number of PEs. In designing accelerators, to reduce the number of data transmissions between PEs, data is accumulated within a single PE. As the number of parallel multiplications in the DSP increases, the number of result accumulation paths also increases. This leads to higher register consumption. Consequently, at this point, a single PE consumes more logic resources. When the overall design resources remain constant, the number of PEs that can be designed decreases. The level of parallelism within a PE and the number of PEs are crucial factors affecting the computational power of the accelerator. Current works have not focused on the relationship between data packing methods within the DSP and the overall computational power of the accelerator.

2.2 Unleashing the Potential of FPGA Heterogeneous Resources

In recent years, researchers have started to explore the development of additional computing engines using the remaining LUT resources, aiming to further enhance the performance of FPGA-based DNN accelerators [9, 12, 32, 40]. Therefore, reducing the logic resource consumption of high-performance mixed-precision PEs based on LUTs has become a research hotspot. These efforts can be broadly divided into two aspects: the divide-and-conquer parallel strategy and the bit-serial strategy. The former realizes mixed-precision computation by designing parallel small multipliers with different architectures to form large multipliers, while the latter achieves mixed-precision computation through bit-by-bit calculation with different time overheads. Each aspect includes

works in 1D and 2D dimensions. The former is internally divided into the **sum apart (SA)** and **sum together (ST)** strategies. The SA method stores low-precision multiplication results separately, while the ST method combines results to reduce output bandwidth requirements. The latter is divided into single-bit serial and multi-bit serial. The traditional MAC method with data gating achieves precision scaling by only calculating the most significant bits and setting the least significant bits to zero [24]. However, when the precision is reduced to 4b and 2b, the throughput improvement is minimal [24]. The 1D divide-and-conquer SA adopts a divide-and-conquer strategy, constructing an $8b \times 8b$ multiplier from four $2b \times 8b$ sub-multipliers, followed by a configurable shift-add logic block [33]. The 1D divide-and-conquer ST accumulates parallel-generated results instead of storing them separately [7]. The baseline $8b \times 8b$ multiplier of 2D divide-and-conquer SA is implemented by $16 \times 2b \times 2b$ multipliers and configurable shift-add logic blocks, supporting multiple precision modes [7]. The 2D divide-and-conquer ST is similar to 2D D&C SA but accumulates parallel-generated products to avoid output bandwidth explosion [31]. Based on BitFusion, subsequent studies have supported inter-layer and intra-layer mixed precision [16]. When using small multipliers to construct large-scale multiplication calculations, FPGA's small logic-based incomplete sub-multipliers propose an asymmetric splitting method [8]. The subword parallel SA is based on a 2D symmetric scalable array multiplier architecture, adopting a top-down design approach [25]. On this basis, sign number processing techniques are proposed through constant vectors and sign extension, minimizing the overhead of two's complement operations and supporting pipeline optimization [5]. The subword parallel ST is also based on a 2D symmetric scalable array multiplier architecture but activates the array multiplier through reverse diagonal patterns to accumulate all subword results [23]. In 1D bit-serial, weights are input iteratively at 1-bit, while activations remain full-precision and are sent in parallel [17]. In 1D multi-bit serial, weights are input serially at 2 or 4 bits, reducing the number of iterations [17]. The 2D bit-serial extends 1D bit-serial to a 2D scalable architecture, feeding weights and activations bit-serially [30]. In 2D multi-bit serial, weights and activations are input serially at 4 bits, significantly shortening the computation cycle [30]. Compared with 1D bit-serial, 1D multi-bit serial reduces clock cycles and energy consumption (e.g., 4-bit serial energy consumption is 1.5 times the baseline) and improves throughput, but precision scaling relies on data gating [4]. M4BRAM integrates bit-serial computation with parallel multiplication to scale MAC latency with precision linearly. It reduces memory access conflicts and latency through collaborative BRAM-computation design [10]. MSD designs multipliers using a bit-serial approach and optimizes the encoding method [40]. N3H further reduces the idle computing resources caused by the computational time gaps between heterogeneous engines through asynchronous instructions and methods for balancing heterogeneous and new workloads [12].

Although these studies have made progress in improving computational efficiency, most of them do not fully consider the computational characteristics of mixed-precision quantized neural network models. Analyzing the computation proportion of different bitwidth combinations in the model is crucial for designing high-computing-power accelerators to deploy such models.

2.3 DSP Structure-Guided Mixed-Precision Quantization

MPQ employs representations of different numerical precisions in various parts of a model. This approach can significantly reduce computational and storage requirements while maintaining model performance, thereby enhancing the efficiency of deploying deep learning models [15]. Given the high demands of MPQ algorithms for hardware support and the limitations of existing hardware implementations, developing dedicated accelerators using FPGAs has emerged as an ideal way to optimize the execution efficiency and flexibility of these algorithms [35]. This enables the achievement of a balance between high performance and low power consumption while ensuring computational accuracy [6]. Current MPQ algorithms aim to minimize precision loss while achieving

lower computational complexity and storage requirements. However, they often overlook the hardware characteristics of different devices. For instance, FILM proposes a more fine-grained intra-layer MPQ to replace the previous coarse-grained uniform low-precision quantization and inter-layer MPQ. Nevertheless, only FPGA-based accelerators can efficiently deploy this algorithm [36]. MPQ introduces an MPQ search algorithm based on the w and dynamic programming, which comprehensively considers hardware constraints and accuracy requirements [42]. This work demonstrates that hardware guidance can effectively improve the accuracy of hardware inference and identify more suitable model quantization schemes for hardware. However, this work mainly focuses on accelerator architectures based on RISC-V processors and does not consider the DSP structure of FPGAs. Reducing the data bit width does not necessarily enhance the parallel computing capabilities of DSP-based FPGA accelerators. Instead, it may lead to a decline in the accuracy of neural network models.

3 Methodology

This section presents the principal innovative approaches of the HMSA framework from the perspectives of algorithm design and hardware-software co-design. First, we comprehensively analyzed the DSP architecture. By optimizing the utilization of the input bit-width of the DSP, we significantly enhanced the parallelism of multiplication operations within the DSP. Building on this analysis, we introduced a novel resource-computing power code model. This model effectively guides the optimized packing process of DSP resources. Subsequently, we proposed a heterogeneous **systolic array (SA)** architecture for mixed-precision computing. This architecture efficiently utilizes both DSP and LUT resources. Finally, we incorporated hardware-aware guidance into the search process for MPQ bit widths. This approach not only reduced the search space of bit-width combinations but also improved the model's accuracy under the same hardware constraints and computational speed requirements.

3.1 DSP Packing Optimization Strategy Based on Resource-performance Cost Model

In FPGA-based large-scale parallel multiplication computations, the primary reliance is on DSP resources for implementation. On different FPGAs from various manufacturers, the types of DSPs vary. However, the method of DSP data packing under discussion is universal. Therefore, we take the DSP48 in Xilinx's xc7vx690t as an example. The input of Port A is 25 bits, the input of Port B is 18 bits, and Port C supports a 48-bit input. A and B are the two inputs of the multiplier in the DSP. Its output port, Port P, supports a 48-bit output. Prior research has proposed methods combining multiple multiplicand inputs into a single DSP block to efficiently utilize these DSP resources, enabling a single DSP unit to perform multiplications for multiple data points.

3.1.1 Flexible Packing Optimization based on Input Optimal Bit-width Utilization of DSP. In FPGA applications, optimizing the bit-width utilization of DSP slices is essential for enhancing computational performance and resource efficiency. We propose an input optimal bit-width packing method aimed at maximizing bit-width utilization. This method increases DSP parallelism under the same bit-width combination. By constructing a resource-performance cost model, we use the ratio of computational power to resource consumption as the key metric to determine the most efficient packing method and data path design. This approach maximizes computational power while minimizing resource usage. Figure 2 illustrates this innovative packing method. We detail the design concept, implementation process, and significant advantages.

We propose an optimal bit-width data packing method to achieve efficient parallel computation in DSP. This method is based on the principle that the bit-width of the product of two numbers a_i

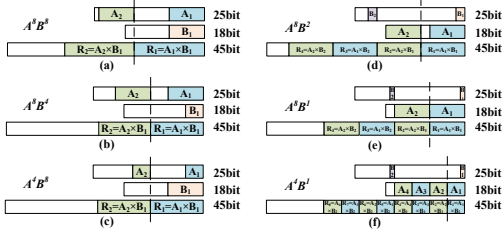
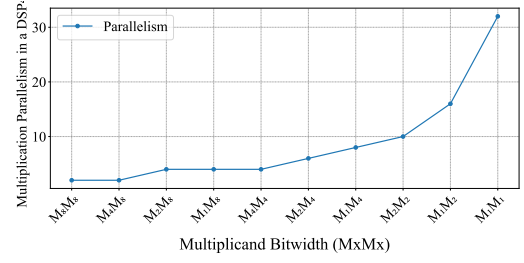


Fig. 2. DSP packing method.

Fig. 3. Relationship between multiplication parallelism and computational bit-width in DSP ($M_{x1}M_{x2}$ denotes two multiplicands with bit-widths x_1 and x_2).

and b_j equals the sum of their respective bit-widths, as illustrated in Equation (1).

$$\text{width}(a_i \times b_j) = \begin{cases} \text{width}_{a_i} + \text{width}_{b_j}, & \text{if } \text{width}_{a_i} > 1 \text{ and } \text{width}_{b_j} > 1 \\ \text{width}_{a_i} + \text{width}_{b_j} - 1, & \text{if } \text{width}_{a_i} = 1 \text{ or } \text{width}_{b_j} = 1 \end{cases} \quad (1)$$

Specifically, for data a_1 and b , we prepend width_b zeros to a_1 and further concatenate a_2 ahead of it to enable parallel computation of $a_1 \times b$ and $a_2 \times b$. The results are distributed in the DSP output as follows: the result of $a_1 \times b$ spans from 0 to $\text{width}_{a_1} + \text{width}_b - 1$, while the result of $a_2 \times b$ spans from $\text{width}_{a_1} + \text{width}_b$ to $\text{width}_{a_1} + \text{width}_b + \text{width}_{a_2} + \text{width}_b - 1$, as shown in Figure 2(a). When prepending additional data (e.g., b_2) to b_1 , we pad $\text{width}_{a_1} + \text{width}_b + \text{width}_{a_2}$ zeros before b_1 and then concatenate b_2 . The results are distributed as follows: the result of b_1 spans from 0 to $\text{width}_{a_1} + \text{width}_{b_1} - 1$, and the result of b_2 spans from $\text{width}_{a_1} + \text{width}_{b_1}$ to $\text{width}_{a_1} + \text{width}_{b_1} + \text{width}_{a_2} + \text{width}_{b_2} - 1$, as shown in Figure 2(d). Experimental results indicate that placing larger bit-width data at port b can more efficiently utilize the DSP input port's bit-width resources, thereby improving overall computational efficiency. The position range of the n -th number can be expressed as:

$$[(n-1) \cdot (\text{width}_a + \text{width}_b) : n \cdot (\text{width}_a + \text{width}_b) - 1], \quad (2)$$

where width_a and width_b represent the bit-widths of data a and b , respectively, and n denotes the index of the number.

We observed that the bit-width combinations of the multiplicands affect the multiplication parallelism within a single DSP, thereby influencing the computational parallelism in PEs. However, under certain precision settings, the level of parallelism remains consistent even when the input data bit-widths vary. Based on this observation, we systematically analyzed and summarized the multiplication parallelism within a single DSP after data packing for various bit-width combinations. The relationships are illustrated in Figure 3.

When performing different numbers of multiplications within a single DSP, the programmable logic resources consumed by the corresponding accumulators vary. Specifically, for a multiplication parallelism of two, as shown in Figure 4(a), the logic resources required for result accumulation can be expressed as $R_2 = (\text{width}_a + \text{width}_b + \text{width}_{\text{extra}}) \times 2$, where width_a and width_b represent the bit-widths of input data a and b , respectively, and $\text{width}_{\text{extra}}$ denotes the additional bit-width required during accumulation. Similarly, for a multiplication parallelism of four, as shown in Figure 4(b), the logic resources required are given by $R_4 = (\text{width}_a + \text{width}_b + \text{width}_{\text{extra}}) \times 4$. Higher multiplication parallelism leads to significantly greater resource consumption; for example, R_4 is twice that of R_2 . Given the limited resources available for each PE, higher resource consumption results in fewer PEs being implementable. Consequently, to optimize the data path within the

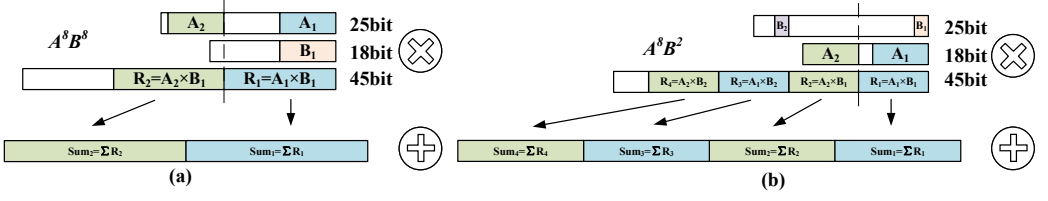


Fig. 4. Relationship among the logic resources consumed by the accumulator, the multiplication parallelism, and the multiplicand bit-widths.

DSP and the design of PEs, it is essential to establish a resource-performance model for **Design Space Exploration (DSE)**. This model establishes the relationship between resource consumption and computational capability under various configurations, facilitating the selection of the optimal design to maximize accelerator performance.

3.1.2 Resource-performance Cost Model. To maximize the utilization of DSP resources, we propose an optimal bit-width utilization method. Our study reveals that the parallelism within the DSP varies significantly under different data path schemes, which can be expressed as $P_{\text{dsp_pe}} = f(S)$, where $P_{\text{dsp_pe}}$ represents the parallelism of the DSP PE, and S denotes the data path scheme (e.g., the design of input data packing and output accumulator allocation in DSP slices). The final selected set of possible data path schemes is represented by $\mathcal{S} = \{\text{M8M8, M8M4, M8M2, M8M1, M4M4, M4M2, M4M1, M2M2, M2M1, and M1M1}\}$. For example, Figure 2(a) corresponds to the M8M8 data path, Figure 2(d) corresponds to the M8M2 data path, and (d) corresponds to the M4M4 data path.

We define the resource consumption model for DSP PE as follows: $\text{Resource}_{\text{dsp_pe}} = f(S)$, where S represents the data path scheme. As the parallelism varies, different data path designs emerge. However, the more data paths supported by the accelerator's PE, the higher the resource consumption. Therefore, determining the optimal combination of data paths to achieve the highest computational performance is crucial.

To evaluate the performance of various data path schemes, we experimentally measured the resource consumption of all selected schemes in terms of DSP PE and constructed a computational core to determine its resource consumption, parallelism, and operating frequency. Based on these measurements, we calculated the computational capability P_{comp} under different data path schemes using the following formula:

$$P_{\text{comp}} = \sum_{(x_1, x_2) \in \mathcal{D}} \frac{L_{x_1, x_2}}{N_{\text{lut_pe}} \times P_{\text{lut_pe}} + N_{\text{dsp_pe}} \times P_{\text{dsp_pe}}}, \quad (3)$$

where $\mathcal{D} = \{(x_1, x_2) \mid x_1, x_2 \in \{1, 2, 4, 8\}, x_1 \geq x_2\}$ represents the full set of weight and activation bit-width combinations. Here, (x_1, x_2) denotes the bit-width combination of weights and activations (e.g., $W8A8$ indicates a weight bit-width of 8 and an activation bit-width of 8). L_{x_1, x_2} represents the computational load (which represents the number of computational operations) for each precision combination, $N_{\text{lut_pe}}$ and $P_{\text{lut_pe}}$ denote the number of logic PEs and their parallelism, respectively, while $N_{\text{dsp_pe}}$ and $P_{\text{dsp_pe}}$ represent the number of DSP PEs and their parallelism, respectively.

To ensure that the design remains feasible within resource constraints, the total resources of the computational core must satisfy:

$$\text{Resource}_{\text{comp_core}} > N_{\text{dsp_pe}} \times \text{Resource}_{\text{dsp_pe}}, \quad (4)$$

where $\text{Resource}_{\text{comp_core}}$ represents the total resource of the computational core, $N_{\text{dsp_pe}}$ is the number of DSP PEs, and $\text{Resource}_{\text{dsp_pe}}$ is the resource consumption of each DSP PE. Additionally, the logical resource constraint must satisfy:

$$\text{Resource}_{\text{lut_rem}} > N_{\text{lut_pe}} \times P_{\text{lut_pe}} \times \text{Resource}_{\text{pe}}, \quad (5)$$

where $\text{Resource}_{\text{lut_rem}}$ represents the remaining logical resources, $N_{\text{lut_pe}}$ is the number of logic PEs, $P_{\text{lut_pe}}$ is the parallelism of each logic PE, and $\text{Resource}_{\text{pe}}$ is the resource consumption of each logic PE.

The input to our model includes the computational load of the model under all precision combinations (e.g., the computational load for $W_{x_1} A_{x_2}$) and the value space of x_1 and x_2 . The output is the optimal data path selection within the DSP (e.g., M8M8, M8M2). Our cost model-guided DSP packing method achieves higher actual computational performance for the accelerator, resulting in faster model inference speeds.

3.2 Adaptive Selection of LUT-Based Computing Architectures via Mixed-Precision Model Bitwidth Distribution and Performance Cost Model

3.2.1 Heterogeneous SA Computing Engines based on DSP and LUTs. When deploying neural network accelerators on embedded FPGA platforms, design resources are often severely limited. To achieve higher computational throughput and accelerate model inference within these constraints, existing research commonly employs the approach of constructing heterogeneous computing engines using the remaining logic resources [12, 40]. To fully utilize FPGA resources in designing a high-performance mixed-precision neural network accelerator, we propose a mixed-precision SA accelerator architecture featuring heterogeneous computing engines. The computing engine is divided into D-MAC and L-MAC components. The D-MAC is a multiply-accumulate unit designed based on DSP resources, while the L-MAC is a multiply-accumulate unit constructed using logic resources. In Section 3.1, we proposed that a MPQ cost model should guide the design of DSP engines to optimize performance. In Section 3.2, after completing the design of the DSP computing core and other hardware modules of the accelerator, we utilize the remaining logic resources to implement the L-MAC, aiming to optimize overall computational efficiency.

In the design of LUT-based mixed-precision computational units, existing research predominantly focuses on optimizing logic resources for general mixed-precision multiply-accumulate computations, while overlooking the unique computational characteristics of mixed-precision quantized neural network models [4, 7, 8, 16, 17, 30, 31, 33]. However, in existing technologies, the implementation of mixed-precision computing typically relies on operations such as combination, accumulation, and shifting of calculation results. Additional consumption of logic resources for control and accumulation circuit components is introduced, which limits the computational parallelism of PEs. In contrast, circuits supporting only a single precision combination have the simplest structure and can achieve the maximum computational throughput under given resource constraints. Nevertheless, this solution has significant limitations: it can only handle computational tasks of specific precision combinations and has zero computing capability for other precision combinations. We define the above-mentioned hardware implementation methods capable of supporting different precision combinations as computing architectures with different precision support types. In particular, when specific precision-bitwidth combinations account for a high proportion of model computations, significant potential for performance improvement may be wasted.

We argue that the selection of the LUT-based computational architecture should be driven by the proportion of computations associated with different precision-bitwidth combinations within the model. This approach ensures that the LUT computational engine is optimally tailored for the

deployment of the specific algorithm, thereby maximizing computational efficiency and resource utilization.

3.2.2 Dynamic Throughput Mapping-Driven LUT Architecture Selection for Model Deployment.

To address this issue, this article proposes an adaptive architecture selection method based on the precision distribution characteristics of the model. This method accurately determines the bitwidth combinations that the LUT computing engine should support by quantitatively analyzing the computational performance of different computing architectures under specific precision combinations. Furthermore, based on the selected bitwidth combinations, a high-throughput computing architecture is constructed to maximize the utilization of remaining LUT resources.

For a target neural network model $\mathcal{M}$, its computational graph operations are represented as a set of precision-bitwidth combinations $\mathcal{O} = \{o_1, o_2, \dots, o_n\}$, where each operation o_i corresponds to a tuple (W_i, A_i) of weight precision W_i and activation precision A_i . The precision distribution of the model is characterized by the operation computation proportion vector $\mathbf{f} = [f_{(W_1, A_1)}, f_{(W_2, A_2)}, \dots, f_{(W_q, A_q)}]^T$, where $f_{(W_k, A_k)}$ denotes the proportion of the computation amount of the precision combination (W_k, A_k) in the total model computation, with $\sum_{k=1}^q f_{(W_k, A_k)} = 1$.

The set of computing architectures is defined as $\mathcal{A} = \{a_1, a_2, \dots, a_k\}$, including single-precision architectures $a_i^{\text{single}} = (W_i, A_i)$ (e.g., W1A8, W2A8) and mixed-precision architectures $a_i^{\text{mixed}} = \{(W_{i1}, A_{i1}), (W_{i2}, A_{i2}), \dots, (W_{il}, A_{il})\}$ (e.g., W1A1/W1A8). The computational power response function of architecture a_i for precision combination $p \in \mathcal{P}$ is $C_{i,p} = \mathcal{F}(a_i, p, R_{i,p})$, where $R_{i,p}$ is the unit logical resource amount required for architecture a_i to support precision combination p , and this function is obtained through experimental measurement, representing the operations per second under fixed resources.

To optimize the allocation of remaining LUT resources R_{remain} , we define the search space $\mathcal{P}$ for computing architecture types supporting different precision combinations as:

$$\mathcal{P} = \mathcal{P}_{\text{single}} \cup \mathcal{P}_{\text{mixed}}, \quad (6)$$

where $\mathcal{P}_{\text{single}}$ represents all possible single-precision configurations (e.g., 8-8, 4-4), and $\mathcal{P}_{\text{mixed}}$ represents all possible combinations of single-precision configurations. Let the set of single-precision configurations be $\{p_1, p_2, \dots, p_n\}$, then $\mathcal{P}_{\text{mixed}}$ can be expressed as:

$$\mathcal{P}_{\text{mixed}} = \left\{ \bigoplus_{i \in S} p_i \mid S \subseteq \{1, 2, \dots, n\}, |S| \geq 2 \right\}, \quad (7)$$

where $\bigoplus$ represents the concatenation operation of multiple precisions, and S is any non-empty subset of the index set $\{1, 2, \dots, n\}$ with cardinality $|S| \geq 2$.

Based on the search space $\mathcal{P}$, we design computing architecture hardware supporting different precision combinations using the same logical resources. Through actual measurement, we obtain the computational power data of each precision combination under different precision combination computing architectures for unit logical resources. Then, a LUT $\mathcal{T}$ is constructed to record the measured computational power $C_{i,p}$ of each precision combination $p \in \mathcal{P}$ under different computing architectures $a_i \in \mathcal{A}$.

The goal is to solve for the optimal architecture $a^* \in \mathcal{A}$ and the corresponding resource allocation R_{a^*, p^*} to maximize the total effective computational power:

$$\max_{a \in \mathcal{A}, R_{a,p}} \sum_{p \in \mathcal{P}} f_p \cdot C_{a,p} \cdot \frac{R_{a,p}}{R_{a,p}^{\text{unit}}}, \quad (8)$$

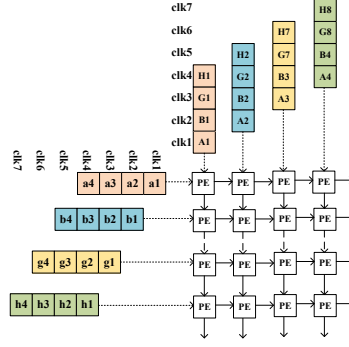


Fig. 5. SA architecture and its data inflow schematic.

subject to the constraints

$$\sum_{p \in \mathcal{P}} R_{a,p} \leq R_{\text{remain}}, \quad R_{a,p} \geq 0, \forall p \in \mathcal{P}, a \in \mathcal{A}, \quad (9)$$

where $R_{a,p}^{\text{unit}}$ is the resource amount required per unit computational power for architecture a to support precision combination p . This problem can be simplified as:

$$a^* = \arg \max_{a \in \mathcal{A}} \sum_{p \in \mathcal{P}} f_p \cdot \eta_{a,p} \cdot R_{a,p}^{\text{max}}, \quad (10)$$

where $\eta_{a,p} = \frac{C_{a,p}}{R_{a,p}^{\text{unit}}}$ is the resource efficiency, and $R_{a,p}^{\text{max}}$ is the maximum allocable resource amount for architecture a to support precision combination p .

Once the optimal architecture a^* and precision combination p^* are determined, resources are allocated to the LUT-based computing unit according to R_{a^*,p^*} , and the computing core is generated using predefined templates. The parallelism P_{LUT} and the resource consumption N_{LUT} of each L-MAC instance are then determined. Specifically, if the resource amount required per unit computational power for the selected architecture a^* to support precision combination p^* is $R_{a^*,p^*}^{\text{unit}}$, then $P_{\text{LUT}} = \frac{R_{\text{remain}}}{N_{\text{LUT}}}$, where N_{LUT} can be derived from $R_{a^*,p^*}^{\text{unit}}$ and architecture design parameters.

3.2.3 Systolic Arrays Architecture. In the implementation of large-scale parallel computing, the BRAM ports connecting caches to computing units may require extensive fan-out routing during layout design. This complexity significantly increases wiring overhead, leading to larger area consumption, reduced achievable frequency, and degraded computational performance. M4BRAM [10] addresses this issue by enabling near-memory computation between BRAM and DSP units, assigning dedicated caches to DSP resources to mitigate shared cache fan-out. However, its weight reuse mechanism, reliant on replicating shuffle networks, is constrained by BRAM port bandwidth, resulting in limited reuse efficiency. In contrast, SAs inherently achieve higher weight reuse rates through their structured dataflow patterns, minimizing memory access bottlenecks while maximizing hardware utilization and computational throughput. SA facilitates the smooth flow of inputs and outputs within its structure, thereby minimizing the need for cache accesses and significantly reducing memory access conflicts during data reuse. Additionally, this architecture simplifies instruction complexity and alleviates scheduling challenges. To further streamline control logic, we adopt a output-Stationary SA architecture.

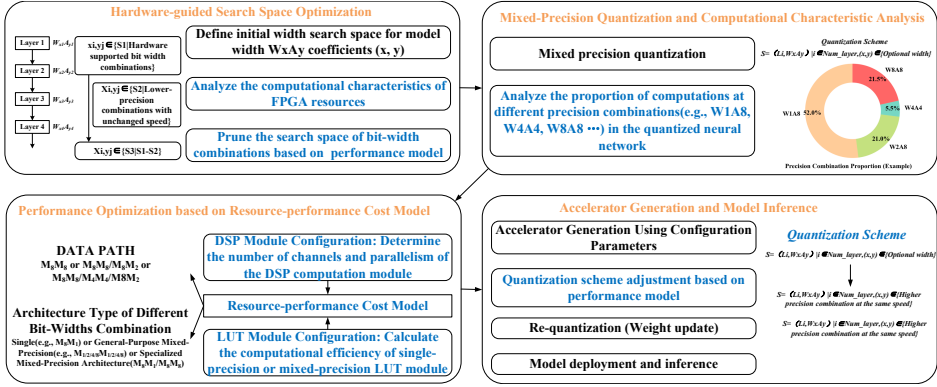


Fig. 6. Workflow of HMSP.

3.3 FPGA-friendly Hardware-guided Bit-width Combination Selection Method

To achieve the co-design of quantization and hardware, we propose a hardware-guided mixed-precision bit-width selection method, as shown in the Figure 6. This co-design process is first proposed in this article, and the parts marked in blue font represent the innovations of this work. Specifically, the method begins with the search optimization for MPQ. Subsequently, within the optimized search space, we determine the quantization precision for each layer of the neural network, namely the bit-widths of weights and activations. This step is referred to as the MPQ process. We define the search space of weight and activation bit-width combinations ($W \times A_y$) based on the hardware-supported configurations. Since the core computational capability of the accelerator is provided by the DSP, the performance model of the DSP serves as the primary reference for the initial pruning of bit-width schemes.

To minimize model accuracy loss while maintaining computational speed, this article proposes a hardware-guided bit-width selection optimization method based on the structural characteristics of DSP slices in FPGAs. Building upon the original mixed-precision search space, we prune bit-width combinations that exhibit identical computational speed but result in significant model accuracy degradation. During DNN inference, activation data is typically represented by A_x , and weight data by W_x , where $W_{x_1}A_{x_2}$ denotes a quantization configuration with a weight bit-width of x_1 and an activation bit-width of x_2 . For a given operator, we refer to its quantization configuration as the *bit-width combination*. In Section 3.1, we observed that different bit-width combinations do not exhibit a strictly linear relationship in terms of multiplication parallelism when using DSPs for parallel computation. For instance, despite differing configurations, W_4A_4 , W_8A_2 , and W_8A_1 achieve the same multiplication parallelism within a single DSP. Additionally, through techniques such as parameter prefetching, Ping-Pong buffering, and pipelining, these bit-width combinations exhibit identical computation times without introducing additional latency during inference. However, although these three bit-width combinations yield the same computational performance, their impact on model accuracy during network quantization varies significantly. Based on the above findings, we systematically analyzed all possible bit-width combinations and grouped them according to their configurations and multiplication parallelism. For the bit-width combinations sharing the same parallelism level, we evaluated their respective model accuracy. We then selected the bit-width combination with the minimal accuracy degradation as the optimal choice, while pruning those combinations with significant accuracy loss from the original search space. This approach effectively reduces unnecessary data paths and significantly improves the accuracy performance of the quantization algorithm when deployed on FPGAs. For instance, W7A8

and W8A8 exhibit the same computational efficiency in DSP packing. Given that 7-bit data is typically stored in an 8-bit format, W7A8 can be removed from the search space. Similarly, to reduce search complexity, schemes such as W4A8, W5A8, and W6A8 are also eliminated from the search space.

In the MPQ process, we employ the simulated annealing method for searching, which was proposed by MBFQuant [28]. The main contribution of this article lies in the co-design optimization of hardware and quantization, emphasizing the use of the DSP hardware computational characteristics of the FPGA to guide the search space of selectable bit-width combinations during MPQ. After completing the MPQ, we analyze the computational characteristics of the MPQ model to guide the DSP computational engine D-MAC in selecting the data path design method with the highest computational power, thereby designing the DSP computational engine (refer to Section 3.1). Based on the computational characteristics of the model, we guide the LUT computational engine in selecting the computational architecture type with the highest computational power (refer to Section 3.2). The bit-width search space can be adjusted according to the precision requirements of specific applications. For illustration, the experimental part of this article uses W and A bit-widths that are set to 1/2/4/8 bit for testing and evaluation. Based on the resource-computational cost model, the hardware configuration of the accelerator can be completed accordingly. Then, we further analyze the compatibility between the MPQ algorithm and the resource-computational cost model. Since the computational architecture of the DSP computational engine is selected based on the model's computational characteristics and design resources, it can efficiently support specific precision combinations (e.g., W1A8, W8A8). However, other non-compatible precision combinations (e.g., W4A4) need to be calculated by padding with zeros. To address this issue, we use the resource-computation cost model and leverage the accelerator's parallel architecture to adjust the bitwidth configurations to the maximum precision combinations with identical computational efficiency. After determining the final bit-width combinations of different layers in the model, we re-quantize the model using the bitwidth combinations calculated by HMSP, and deploy the quantized model to the FPGA accelerator.

4 Accelerator Architecture

4.1 Architecture Overview of HMSA

We present the architecture of HMSA, which consists of three key components, as illustrated in Figure 7. The control module is responsible for receiving instructions and performing critical tasks such as tile partitioning, loop unrolling, ping-pong scheduling, and memory management, ensuring efficient coordination of computational resources by dynamically adjusting tile sizes and managing dataflow between different processing stages. To minimize the impact of frequent off-chip memory accesses on overall inference latency and to enhance throughput via pipelined operations, the proposed architecture allocates dedicated ping-pong-based input/weight/result buffers for each compute core. Weights and activation data are fetched into these dedicated buffers through **Direct Memory Access (DMA)** and subsequently transferred to the compute core for processing. Once the computation is complete, the results stored in the result buffer are written back to DDR as activation data for subsequent layers. After dividing the operators executed by the accelerator into tiles, we designed a pre-fetching mechanism for computational parameters, loading the activations and weights of the next tile to be computed into the ping-pong buffers, ensuring uninterrupted data flow and maximizing computational efficiency. Furthermore, we designed a hybrid-precision acceleration engine based on heterogeneous resources such as DSPs and LUTs to accelerate matrix multiplication-like operators. The architecture of this engine follows the optimal design dimensions derived in Sections 3.1 and 3.2, specifically developing a SA architecture template and a PE structure template that can generate the acceleration engine according to the optimized design dimensions.

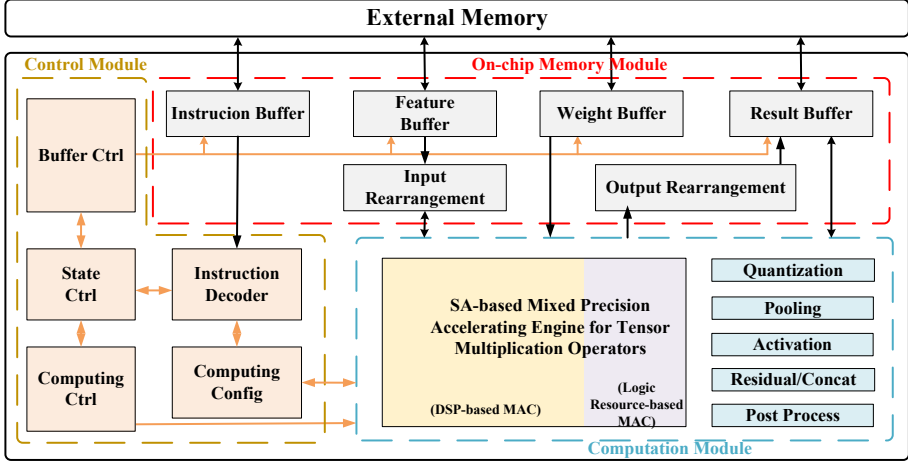


Fig. 7. Architecture overview.

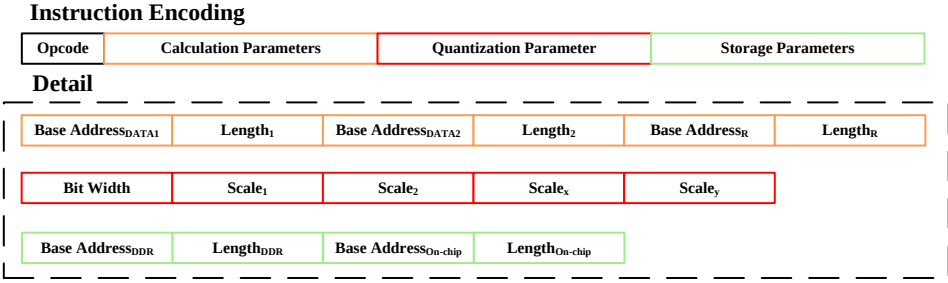


Fig. 8. Instruction structure.

Additionally, we implemented dedicated modules for quantization, pooling, activation, residual connections, concatenation, and post-processing to handle other types of operators in neural networks, which ensures comprehensive support for diverse neural network computations while maintaining high performance and flexibility.

4.1.1 Instruction Design and Instruction Parsing. In the process of neural network inference, the model utilizes a neural network compiler to decompose the computational graph into a series of operator units supported by the accelerator. This process ultimately converts the computational graph into an instruction sequence compatible with the accelerator, where each instruction sequence has a fixed length of 512 bits, as shown in Figure 8.

These instructions are categorized into two types: operator execution instructions and data loading instructions. Opcode is used to represent different computation modes, including convolution, pooling, fully connected layers, activation functions, concatenation, and residual connections. Each instruction is followed by information divided into three categories: computation details, quantization parameters, and memory-related information (such as DDR addresses and on-chip cache addresses). Notably, in our design, instead of relying on automatic tile optimization mechanisms for instruction processing, we have implemented a dedicated module within the control unit. This module performs automatic tile partitioning and address generation for instructions. It dynamically adjusts the tile partitioning of instructions based on computation requirements and accurately

generates corresponding memory addresses to ensure efficient data transfer and processing. During model inference, the instruction sequence is loaded into the on-chip instruction cache, and the dataflow state machine is activated to fetch instructions and relevant data on demand. Additionally, a prefetching mechanism based on the ping-pong operation is employed to proactively prepare the required instructions and data, thereby avoiding computational stalls and further enhancing overall performance. This design not only optimizes the utilization of hardware resources but also significantly improves the efficiency and accuracy of model inference.

4.1.2 Optimization of Quantized Inference. The methodology for quantizing weights to INT8 involves defining the quantized weight value as w_q , the real-valued weight as w_f , the weight scaling factor as s_w , and the quantization zero point of the weight as z_w , where the relationship between these variables is given by $w_f = s_w \times w_q - z_w$; for input feature maps, let the quantized value be x_q , the real value be x_f , and the input feature map's scaling factor be s_x , thus we have $x_f = s_x \times x_q$; to facilitate computation, there exists a relationship among the scaling factors for input feature maps, weights, and biases expressed as $s_{wb} = s_w \times s_x$; regarding output feature maps, denote the quantized value as y_q , the real value as y_f , and the output feature map's scaling factor as s_y , therefore $y_f = s_y \times y_q$; additionally, from the convolution computation formula, the real value of the output feature map can be represented as $y_f = w_f \times x_f + b$; substituting w_f , x_f , and b into this equation yields $y_f = s_w \times w_q \times s_x \times x_q - z_w \times s_b \times b$; substitute s_{wb} into the above equation and factor out common terms to obtain $y_f = s_{wb} \times w_q \times x_q - z_w \times x_q + s_b \times b$; substituting y_f back into the equation gives $s_y \times y_q = s_{wb} \times w_q \times x_q - z_w \times x_q + s_b \times b$; thus $y_q = \frac{s_{wb}}{s_y} \times w_q \times x_q - \frac{z_w}{s_y} \times x_q + \frac{s_b \times b}{s_y}$; from the above equations, it is deduced that $y_q - \frac{z_w}{s_y} \times x_q + \frac{s_b \times b}{s_y}$ represents the INT32 computation result of the upper convolution layer; here, y_q is the INT8 input of the convolution layer; multiplying the computation result by $\frac{s_{wb}}{s_y}$ and performing quantization inter-layer conversion truncation enables obtaining the input feature map for the next convolution layer without loss of precision; it should be noted that $\frac{s_{wb}}{s_y}$ is an FP32 floating-point number; whether considering the occupation of on-chip computing resources or the computation time, the implementation of FP32 multipliers is unacceptable compared to INT8 multipliers; consequently, the FP32 result of $\frac{s_{wb}}{s_y}$ can be replaced by the multiplication of two INT8 data and shift operations, denoted as m_1 and m_2 , respectively, with their relationship being $\frac{s_{wb}}{s_y} = m_1 \times 2^{m_2}$; therefore, letting the output result of the convolution operator be o , the quantization inter-layer conversion operation can be expressed as $\frac{s_{wb}}{s_y} \times o = m_1 \times o \gg m_2$; here, m_1 and m_2 are INT8 values, and different $\frac{s_{wb}}{s_y}$ correspond to different m_1 and m_2 ; given that shift operations are more favorable for FPGA implementations, the aforementioned equation can be optimized accordingly.

4.2 Conv Computing with Heterogeneous SA

In Figure 9, we present the data flow organization in memory and the convolution process. The data flow does not align with the input requirements of the SA architecture. To address this mismatch, we propose the im2col hardware module, which reformats the input data during the convolution process into a continuous data stream for seamless integration into the SA through specialized hardware, as shown in Figure 10. This design significantly improves the utilization of PEs and enhances the overall performance of the hardware system. The im2col module transforms three-dimensional feature maps and two-dimensional convolution kernels into one-dimensional vectors to facilitate efficient convolution computations. This transformation is achieved through address generation and shift register operations. Specifically, data is sequentially read from the input buffer and written into the shift register array. Once sufficient data has been accumulated to

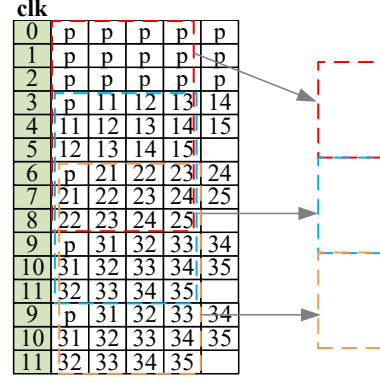
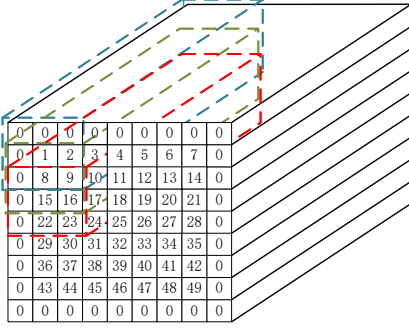


Fig. 9. Data required for convolving feature map tensors. Fig. 10. Feature mapping to SA during im2col.

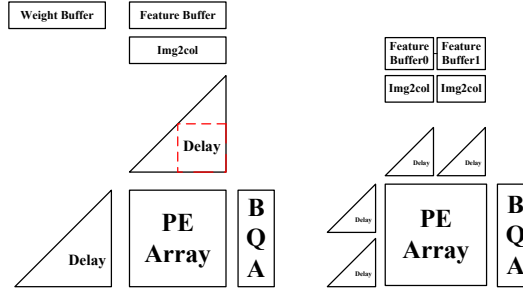


Fig. 11. Optimization of the delay circuit for data inflow in the SA.

satisfy the sliding condition, the shift register begins shifting, generating the required data streams for each output position sequentially. The module can process multiple three-dimensional inputs concurrently and supports convolution computations for kernels of varying sizes (e.g., size 1 or 3). By reorganizing the data flow, this approach significantly enhances both the efficiency and parallelism of convolution operations.

4.3 SA Delay Optimization

Due to the inherent characteristics of the SA, data is organized into streams, with one data element flowing into the array per cycle. As illustrated in Figure 5, the PE located at position (x, y) computes the multiply-accumulate result of the x th row of matrix A and the y th column of matrix B .

Consequently, the time at which the n th data element of the x th row enters the PE at position (x, y_i) is given by $n + i - 1$. Similarly, the time at which the n th data element of the y th column enters the PE at position (x_j, y) is $n + j - 1$. To ensure correct computation, it is necessary to implement a delay mechanism for the data streams before they enter the SA. Specifically, the data stream at position (x_j, y_i) must be delayed by $j - 1$ and $i - 1$ cycles, respectively. However, when data flows out of the buffer, multiple data streams are output within a single cycle rather than a single stream. Therefore, a dedicated delay circuit is required for the data entering the SA, as shown in Figure 11(a). The resource consumption of this delay circuit is directly related to the output dimensions of the buffer; larger dimensions result in a greater area occupied by the delay circuit. To address this issue, we propose an optimized design strategy for the input delay circuit of the SA based on block-wise partitioning, as shown in Figure 11(b).

5 Experiment

In our proposed design, the input buffer is subdivided into finer-grained smaller buffers. This allows us to activate different input buffer segments using simple accumulators, eliminating the need for large-area registers to temporarily store data that has been output from the buffer but is not yet ready to enter the SA. Instead, the enable signal of the corresponding small buffer can be activated at the appropriate time, allowing data from the buffer to flow directly into the SA. This significantly reduces the buffering requirements for the SA's input logic, thereby conserving computational resources. Furthermore, this optimization contributes to enhancing both the peak computational performance and the practical efficiency of the accelerator.

5.1 Experimental Setup

The HMSA proposed in this article is implemented on the Xilinx VC709 evaluation board equipped with the XC7VX690T FPGA. We utilize the Verilog language for the design and rely on the Vivado 2019.1 tool to carry out simulation, synthesis, and implementation. Eventually, the design is programmed into the FPGA. In the hardware environment, the evaluation board, together with the supporting storage devices and data transmission interfaces, is utilized to ensure data storage and transmission. The software environment depends on Vivado 2019.1 and related drivers and testing tools to complete the design and debugging.

In the experiments, multiple classic datasets such as FGCSR-42, Cifar-10, Cifar-100, SSDD, and HRSC2016 are selected, covering image classification and object detection scenarios. A variety of neural network models, including VGG9, VGG16, ResNet-18, and so on., are employed. Optimizers like SGD, Adam, and AdamW are used to comprehensively evaluate the performance of the HMSA. In the network benchmarking, we simultaneously employed traditional **Convolutional Neural Networks (CNNs)** to conduct comprehensive comparisons, thus demonstrating the effectiveness of HMSA. These CNNs cover different kernel sizes (1×1 and 3×3) and strides (1×1 and 2×2).

To ensure the reliability of the results, we conduct multiple repeated experiments, record and analyze them in detail. Meanwhile, a control group is established for comparison with other advanced FPGA accelerators/processors. The HMSA is evaluated from multiple indicators, providing a basis for subsequent optimization.

5.2 Evaluation of Data Packing Optimization Methods for FPGA's DSP

To comprehensively evaluate the performance advantages of the proposed optimized bit-width utilization and packing method, we conducted a series of comprehensive experiments and performed an in-depth analysis. Specifically, in the computational performance speedup tests (as shown in Figure 12), the MSD method only supports fixed combinations of two int8 data. For multiplications involving other bit-widths, it relies on padding higher bits with 0/1, which significantly wastes the input port bit-width resources of DSP slices. Consequently, the computational performance speedup remains relatively low across various bit-width combinations, such as W8A8 and W4A8. Although the MP-OPU method improves parallelism by dynamically packing input port A, it neglects the optimization of port B, resulting in a limited improvement in speedup that still falls short of expectations.

In contrast, the method proposed in this article achieves maximum parallelism for DSP-based multiplications under any bit-width combination. As a result, it demonstrates significantly higher computational performance speedups across all tested bit-width combinations compared to MSD [40] and MP-OPU [39]. Notably, while the MPA method does not improve the packing of input data for DSPs, it addresses the issue that compensation corrections are required for computation results during the packing process—a detail overlooked in previous works. By resolving this issue, MPA reduces resource consumption when designing accelerators based on DSP MAC units.

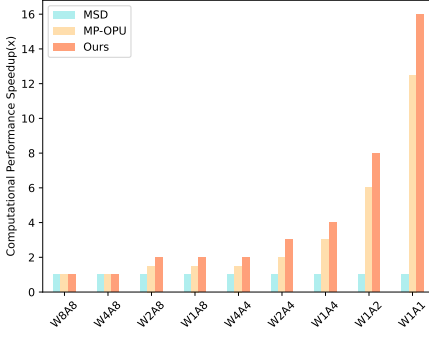


Fig. 12. DSP parallelism capability improvement.

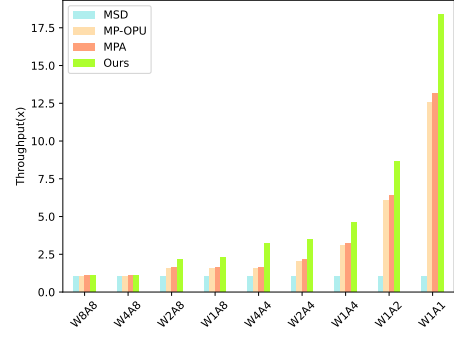


Fig. 13. Throughput speedup of the DSP computing engine.

Therefore, in the throughput experiments (as shown in Figure 13), we implemented accelerators using only DSP-based MAC units based on MSD [40], MP-OPU [39], MPA [26], and our proposed method. Our focus is on evaluating the performance of a DSP computing engine with a single data path. Consequently, the VGG16 model was quantized to a single precision to be deployed on these accelerators for evaluating computational performance. Subsequently, we conducted a detailed comparison of the neural network's inference speed with the throughput achieved by other related works during model inference. Experimental results demonstrate that, due to the efficient utilization of DSP bit-widths, our method exhibits the highest throughput speedup across all bit-width combinations. This fully validates the significant effectiveness of our approach in enhancing both the computational performance of DSPs on FPGAs and the overall performance of the accelerator. The subsequent section will provide a detailed explanation of how to design hardware support for mixed-precision neural networks based on DSP packing methods.

5.3 Performance of Performance Cost Model

5.3.1 Performance of Resource-performance Cost Model for DSP-based Computational Engine.

In the DSP slices, varying bit-width combinations result in different degrees of parallelism for multiplication operations within a single DSP. Given this variation in parallelism, the consumption of logical resources for tasks such as data transmission, data packing, and result accumulation in each PE also differs. An extensive series of experiments were conducted to quantify the logical resource consumption of PEs supporting various data path configurations, with detailed data presented in Table 1. Analysis of the figures indicates that accelerators exhibit distinct computational performance characteristics under different PE data path designs.

Considering the performance discrepancies among PEs with different data path configurations, we implemented a DSP-based accelerator using the Xilinx XC7V690T FPGA and evaluated its peak computational performance across different data bit-widths, as shown in Figure 14. The results demonstrate that as the types of data paths supported by PEs increase, so does the consumption of logical resources, fundamentally explaining why different design schemes have varied peak computational capabilities. Based on the aforementioned experimental data, modeling was performed according to the method described in Section 3.1. By analyzing the computational requirements of different precision combinations in the hybrid-precision model intended for deployment, the optimal configuration choices for PE data paths during accelerator design were determined.

5.3.2 Performance of Bitwidth Distribution and Performance Cost Model for LUT-based Computational Engine. Based on the remaining resources of the DSP-designed accelerator, we further

Table 1. PE Supports Performance Comparison of Different Mixed-precision Data Paths Within DSP48

Mixed Data Path (Multiplicand1 - Multiplicand2)	Logical Resource(PE)		PE Design Parallelism
	LUT	FF	
M8M8	90	100	3,584
M8M1	97	130	3,584
M8M8/M8M2	138	154	3,072
M8M8/M4M4	150	154	2,560
M8M8/M4M4/M8M2	182	154	2,048
M8M8/M4M4/M8M2/M4M1	253	162	1,280
M8M8/M4M4/M8M2/M4M1/M2M1	263	162	1,280
M8M8/M4M4/M8M2/M4M1/M2M1/M1M1	297	162	1,024
M8M8/M4M4/M8M2/M4M2/M4M1/M2M1/M1M1	312	172	1,024

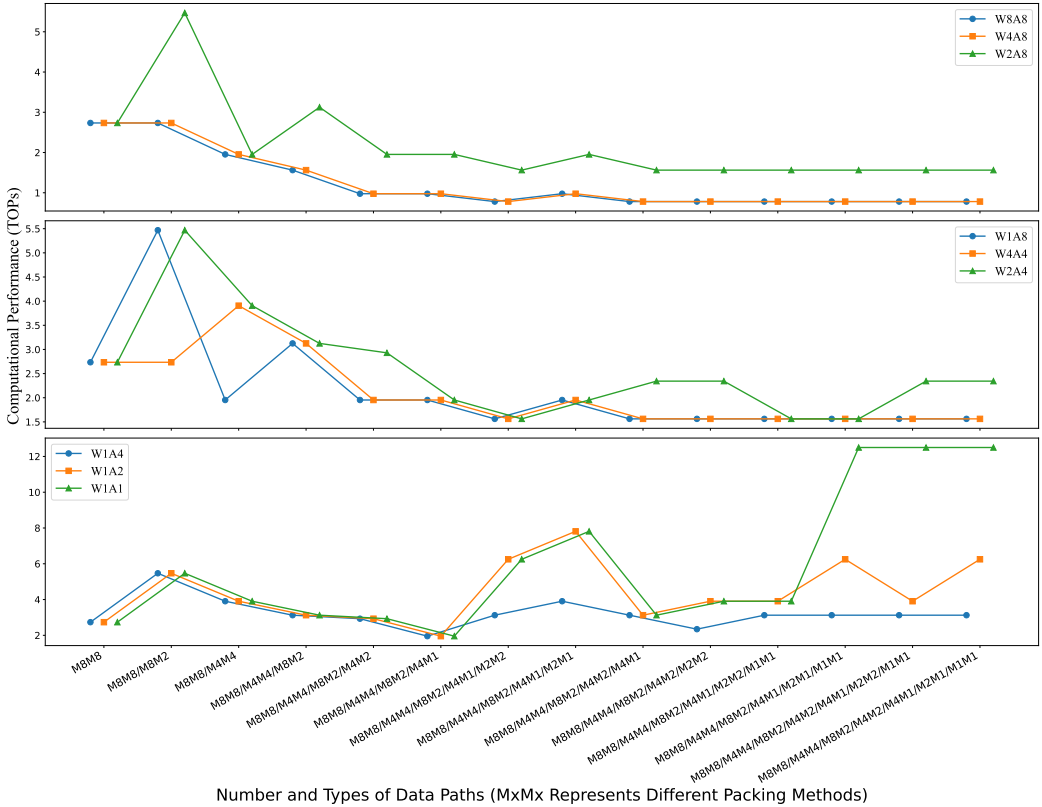


Fig. 14. Relationship between computational performance and data bit-width configurations ($W_{x_1}A_{x_2}$) in a DSP-based accelerator design. The x-axis represents the types of data paths designed within a DSP, which directly influence the parallelism of computations involving different data bit-widths inside the DSP. Specifically, x_1 represents the bit-width of the quantized weights, and x_2 represents the bit-width of the quantized activations.

conducted the design of LUT-based heterogeneous computing engines. To this end, we tested the logical resource consumption of LUT-based heterogeneous computing engines supporting various data paths. The tested data paths encompassed all combinations mentioned in Section 3.2. In Section 2.2, we introduced the current optimized architectures for mixed-precision acceleration units designed based on LUT resources. These architectures are applied to mixed-precision neural networks in various scenarios, where the proportion of computational loads for different bit-width combinations varies. Different architectural designs yield different performance benefits. To quantify the advantages and disadvantages of different architectural designs, we tested various design approaches for mixed-precision computational units. Under the same LUT resource constraints, we generated computational engines using different architectural methods and tested the computing power of these engines under different bit-width combinations, followed by a comparative analysis. The comparison results are shown in Figure 15. It can be observed from the figure that, except in the case of W8A8, the 2D-SA computational unit architecture achieves the highest computing power under the same logical resource constraints. Based on the cost model, we select the architectural type with the highest comprehensive computing power according to the proportion of computational loads for the model's bit-width combinations. Based on the above experimental results, after selecting the optimal LUT mixed-precision computational architecture for our deployment model, we further conducted more precise, in-depth, and fine-grained computational architecture selection. We believe that supporting all bit-width combinations will lead to additional logical resource consumption, potentially resulting in logical resource wastage during model deployment and leaving undiscovered potential for computing power improvement. Therefore, based on the analysis of the computational loads for different bit-width combinations of the model, we proposed a broader search space in Section 3.2, which includes architectures with single bit-width combinations and incomplete mixed-precision bit-width combinations. The technical basis for this approach is that we already have a high-performance DSP computing engine; if the LUT engine does not support computations for certain bit-width combinations, the DSP engine can be used. The role of the LUT engine is to provide the highest computing power for computations with the highest proportion of computational loads under constrained logical resource conditions. Therefore, we used two neural network models with different computational characteristics for testing to demonstrate the effectiveness of the method. The first model is VGG16, with 96.4% W1A8 computations and 3.6% W8A8 computations. The second model is also VGG16, with 60.25% W1A8, 18.07% W4A4, and 21.68% W8A8 computations. We used the convolutional layers of the above networks to test the computing power performance of computational units designed with different architectural types, as shown in Figure 15.

From Model 1 in the Figure 16, it can be seen that when the computational proportion of a certain bit-width combination is high, using an architecture that only supports single precision achieves higher computing power performance than a mixed-precision model. When computing the W1A8 layer, it reaches a peak computing power of 6.385 TOP/s. In contrast, in Model 2, the computing power performance of the single-precision architecture is poor, and at this time, selecting a general mixed-precision computational architecture based on the cost model can demonstrate higher computing capabilities.

5.3.3 Experimental Validation. We evaluated our resource-performance cost model using a series of neural networks quantized with the MBFQuant method, with an allowable accuracy loss set to 2%. The quantizable bit-widths for both weights and activations were configured as 1/2/4/8 bit. Our results demonstrate that the performance of DSP and LUT-based heterogeneous accelerators guided by the cost model has significantly improved. The baseline for our experiments is an accelerator architecture designed without the guidance of a cost model.

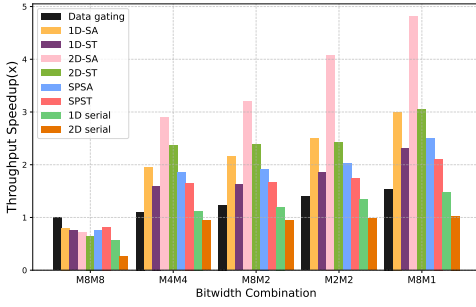


Fig. 15. Performance analysis of computational throughput for different bit-width combinations under diverse architectural types.

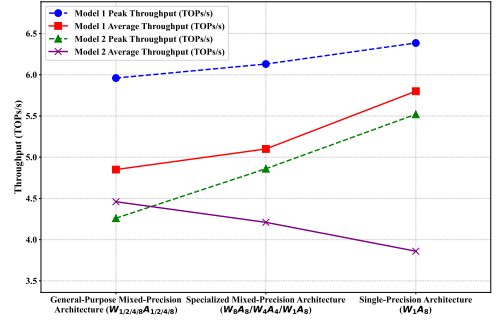


Fig. 16. Throughput performance of mixed-precision quantized models with diverse computational workload distributions across architectures.

Table 2. Network Quantization Speedup Evaluation

Model	Dataset	Optimizer	Speedup		
			INT8	Without cost model	Cost model(Ours)
VGG9	FGCSR-42	SGD	1×	3.12×	5.49×
VGG16	FGCSR-42	Adam	1×	3.24×	5.76×
ResNet-18	Cifar-10	Adam	1×	2.52×	5.61×
ResNet-34	Cifar-100	AdamW	1×	3.36×	5.70×
ResNet-50	FGSCR-42	Adam	1×	3.44×	5.76×
MobileNetV2	Cifar-10	AdamW	1×	2.94×	4.68×
MobileNetV2	FGSCR-42	AdamW	1×	2.94×	4.68×
YOLOv5m	SSDD	SGD	1×	2.76×	5.16×
YOLOv5m	HRSC2016	SGD	1×	2.76×	5.16×
YOLOv5m-MobileNetV2	SSDD	SGD	1×	2.88×	4.98×

In all the tested models, the speedup ratios of using the cost model are significantly higher than those without the cost model, as shown in Table 2. This indicates that the introduction of the resource-performance cost model has a remarkable effect on enhancing the performance of DSP- and LUT-based heterogeneous accelerators. The improvement amplitudes of speedup ratios vary among different models after applying the cost model. For models such as VGG16 and ResNet-50, the speedup ratios reach 5.76× after using the cost model, showing prominent improvement effects. However, for MobileNetV2, the speedup ratio only reaches 4.68×, which is relatively lower.

5.4 Performance Evaluation of Hardware-Guided Inter-Layer Precision Allocation

In DNN quantization, determining the optimal quantization bit-width for each layer is a critical issue. During the process of searching for optimal quantization bit-width combinations across various layers, MBFQuant [28] fails to consider the specific characteristics of the underlying hardware architecture. This oversight can lead to suboptimal bit-width selections, which, in turn, may adversely affect the accuracy and performance of the model. To address these issues, edge-MPQ [42] introduced hardware evaluation metrics to more efficiently explore the MPQ search space

Table 3. Network Quantization Accuracy Evaluation with the Same Inference Speed

Model	Dataset	Optimizer	Top-1 Accuracy				
			FP32	INT8	MBFQuant [28]	Edge-MPQ [36]	HMSP(Ours)
VGG9	FGCSR-42	SGD	84.7%	83.6%	81.8%	82.0%	82.2%
VGG16	FGCSR-42	Adam	92.5%	92.3%	92.1%	92.1%	92.3%
ResNet-18	Cifar-10	Adam	92.3%	91.5%	90.8%	90.6%	91.2%
ResNet-34	Cifar-100	AdamW	72.2%	71.7%	71.4%	71.6%	71.7%
ResNet-50	FGSCR-42	Adam	97.9%	97.2%	95.8%	96.8%	97.0%
MobileNetV2	Cifar-10	AdamW	91.3%	90.6%	89.8%	89.6%	90.5%
MobileNetV2	FGSCR-42	AdamW	96.7%	95.8%	94.8%	95.2%	95.5%
YOLOv5m	SSDD	SGD	70.8%	68.8%	68.4%	69.0%	69.8%
YOLOv5m	HRSC2016	SGD	74.6%	74.4%	73.4%	73.8%	74.4%
YOLOv5m-MobileNetV2	SSDD	SGD	63.3%	61.8%	60.8%	61.1%	61.3%

to find quantization schemes that balance hardware costs and model accuracy. However, their modeling approach, which adjusts the metric wbased on the speed and accuracy of running hybrid DNNs, introduces an evaluation of computational speed influenced by hardware structure but lacks a thorough analysis of the underlying hardware architecture principles.

Our method directly analyzes the structural characteristics of FPGA DSP slice architecture. By eliminating quantization bit-width combinations that do not improve inference speed and may introduce accuracy loss, we ensure that the retained bit-width combinations maximize computational efficiency on the hardware while minimizing accuracy degradation. This approach not only enhances the precision performance of MPQ but also provides new insights into hardware-aware quantization. In other aspects of MPQ (such as model training and quantization factor optimization), we adopt the same methodology as MBFQuant [28], introducing our approach only during the bit-width search phase to optimize bit-width selection. As shown in Table 3, compared to MBFQuant and edge-MPQ, our FPGA-friendly hardware-guided bit-width selection method demonstrates higher accuracy across multiple models and datasets.

For instance, on the VGG9 model with the FGCSR-42 dataset, our method achieves a Top-1 accuracy of 82.2%, surpassing the 81.8% of MBFQuant and the 82.0% of edge-MPQ. Similarly, for the ResNet-50 model on the FGSCR-42 dataset, our method reaches an accuracy of 97.0%, outperforming the 95.8% of MBFQuant and the 96.8% of edge-MPQ. These results indicate that through optimized bit-width selection strategies, our method significantly enhances model accuracy without compromising inference speed.

Furthermore, without optimization of the bit-width combination search space, the search space for the entire network's MPQ scheme will increase exponentially with the increase in the number of layers. This results in an excessively large search space for the model and a prolonged search time, as shown in Figure 17. The HMSP method prunes the bit-width search space of MPQ based on the characteristics of the hardware. Therefore, we adopt the same method as the baseline for obtaining the optimal bit-width combination for each layer. Our focus is on the effectiveness of HMSP in trimming the search space and accelerating the search process after applying its bit-width search algorithm. Additionally, our hardware accelerator, configured based on a cost model, is capable of supporting computations with arbitrary precision ranging from 1 to 16 bits. The bit-width selection space for MPQ models is application and model-specific. To illustrate this, we define three initial bit-width spaces, corresponding to the three figures in Figure 17. Experiments demonstrate that as the number of selectable bit-widths increases, the HMSP method achieves a higher proportion of

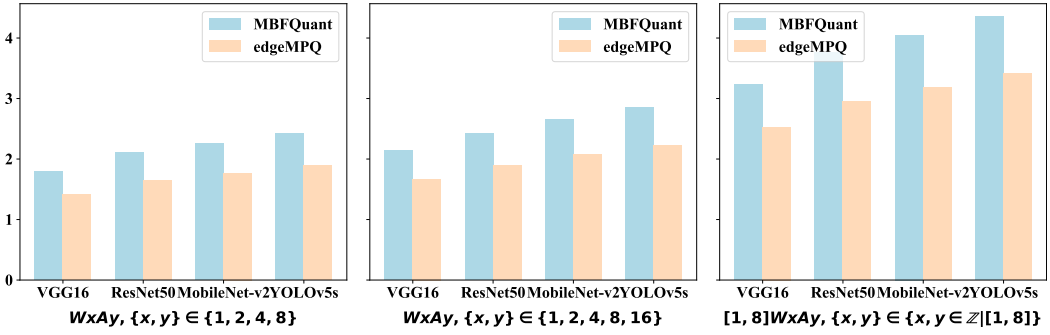


Fig. 17. Acceleration of bit-width combination search. WxAy represents the selectable precision options for the model.

Table 4. Performance Comparison of HMSA with Other Works

	[12]	[36]	[26]	[40]	[39]	Ours	Ours
FPGA	Xilinx	Xilinx	Xilinx	Xilinx	Xilinx	Xilinx	Xilinx
	XC7Z045	ZCU102(ZU9EG)	XC7Z045	ZU9EG	XC7V690T	XC7V690T	XC7V690T
Model	ResNet18	ResNet50	ResNet50	VGG16	VGG16	VGG16	ResNet50
Frequency(MHz)	100	150	200	214	200	200	200
DSP	900(100%)	2,092(83.0%)	840(93.3%)	2,312(91.7%)	3,072(85.3%)	3,328(92.4%)	3,328(92.4%)
LUT(k)	153(70.2%)	181(66.1%)	92.6(42.4%)	152(55.5%)	279(64.4%)	363(83.8%)	363(83.8%)
FF(k)	/	/	9.6(2.2%)	/	324(37.4%)	340(39.3%)	340(39.3%)
BRAM	541(99.3%)	441(48.4%)	/	771(84.5%)	912(62.9%)	1,280(87%)	1,280(87%)
Throughput(TOP/s)	0.447	0.891	0.33	0.588	2.7	4.62	2.88
Latency(ms)	32.47	9.16	/	52.7	11.2	6.65	1.26

search space trimming and a more significant acceleration effect on search time. HMSP effectively reduces per layer's search space and search time for bit-width selection.

5.5 Comparison with the State-of-arts Accelerators

We further compare HMSA with five FPGA accelerators/processors on traditional CNNs, as shown in Table 4. Since quantization methods are beyond the scope of this article, we adopt the networks quantized using HAQ [38], as employed in MP-OPU [39]. However, we enhance the HAQ method by incorporating our proposed FPGA hardware-guided quantization bit-width search space optimization approach. To evaluate the performance of each FPGA processor/accelerator in running the networks, we utilize both throughput and inference latency as metrics. The input image size is set to $224 \times 224 \times 3$.

When evaluating the computational performance of FPGA accelerators, we adopt throughput per DSP (TOP/s per DSP) as a key metric to ensure a fair and unified comparison across different works. This approach effectively eliminates the influence of hardware resource configuration differences, thereby directly reflecting the computational efficiency of each design. Under the same model and operating frequency of 200 MHz, our accelerator demonstrates significantly superior performance compared to prior works. When inferencing VGG16, our accelerator achieves a throughput of 4.62 TOP/s, corresponding to a throughput/DSP of 1.42 GOPs/s/Giga-DSP, which represents an improvement of approximately 57.9% over the previous study [39], which reported 2.7 TOP/s and 0.9 TOP/s/Giga-DSP. This not only highlights higher resource utilization efficiency but also indicates a substantial enhancement in processing capability per unit of DSP resources. Furthermore, our

accelerator reduces the inference latency for the VGG16 model to 6.65 ms, compared to 11.2 ms for MP-OPU [39]. This low-latency characteristic, combined with high throughput, demonstrates that our design excels in resource utilization efficiency and exhibits outstanding real-time processing performance.

6 Conclusion

In this article, we propose a heterogeneous mixed-precision SA accelerator, HMSA. First, an efficient packing method is developed, significantly enhancing the multiplication parallelism within individual DSP slices. Then, we construct a resource-performance cost model to guide the selection of optimal data paths during the design of D-MAC engines. Based on the established resource-performance cost model, we determine the optimal bit-width combinations that maximize inference throughput and select the corresponding L-MAC architecture type. Based on this foundation, we designed heterogeneous computing engines with the SA architecture. During deployment, we propose a hardware-guided MPQ method based on search space pruning, that is, guided by the FPGA's DSP architecture, HMSP. HMSP effectively narrows the search space and eliminates bit-width combinations that fail to improve inference performance but may reduce accuracy. Our work significantly narrows the gap between the actual and theoretical computational speeds of mixed-precision DNNs during practical inference.

References

- [1] Rashed Al Amin, Mehrab Hasan, Veit Wiese, and Roman Obermaier. 2024. Fpga-based real-time object detection and classification system using yolo for edge computing. *IEEE Access* (2024).
- [2] AMD Xilinx. 2024. *7 Series DSP48E1 Slice User Guide (UG479)*. AMD Corporation, Santa Clara, CA. Retrieved from https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1
- [3] Andrew Boutros, Sadegh Yazdanshenas, and Vaughn Betz. 2018. Embracing diversity: Enhanced DSP blocks for low-precision deep learning on FPGAs. In *Proceedings of the 2018 28th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 35–37.
- [4] V. Camus, C. Enz, and M. Verhelst. 2019. Survey of precision-scalable multiply-accumulate units for neural-network processing. In *Proceedings of the IEEE 1st International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. 57–61.
- [5] Vincent Camus, Linyan Mei, Christian Enz, and Marian Verhelst. 2019. Arithmetic core generation using bit heaps. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 4 (Dec. 2019), 697–711.
- [6] Vincent Camus, Linyan Mei, Christian Enz, and Marian Verhelst. 2019. Review and benchmarking of precision-scalable multiply-accumulate unit architectures for embedded neural-network processing. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 4 (2019), 697–711. DOI: <https://doi.org/10.1109/JETCAS.2019.2950386>
- [7] V. Camus, L. Mei, C. Enz, and M. Verhelst. 2019. Review and benchmarking of precision-scalable multiply-accumulate unit architectures for embedded neural-network processing. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 4 (Dec. 2019), 697–711.
- [8] Vincent Camus, Linyan Mei, Christian Enz, and Marian Verhelst. 2019. Small logic-based multipliers with incomplete sub-multipliers for FPGAs. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 4 (Dec. 2019), 697–711.
- [9] Sung-En Chang, Yanyu Li, Mengshu Sun, Runbin Shi, Hayden K-H So, Xuehai Qian, Yanzhi Wang, and Xue Lin. 2021. Mix and match: A novel fpga-centric deep neural network quantization framework. In *Proceedings of the 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 208–220.
- [10] Yuzong Chen, Jordan Dotzel, and Mohamed S Abdelfattah. 2023. M4bram: Mixed-precision matrix-matrix multiplication in fpga block rams. In *Proceedings of the 2023 International Conference on Field Programmable Technology (ICFPT)*. IEEE, 69–78.
- [11] Behnam Ghavami, Mahdi Sajadi, Lesley Shannon, and Steve Wilton. 2024. Boosting multiple multipliers packing on FPGA DSP blocks via truncation and compensation-based approximation. In *Proceedings of the 2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 222–227.
- [12] Yu Gong, Zhihan Xu, Zhezhi He, Weifeng Zhang, Xiaobing Tu, Xiaoyao Liang, and Li Jiang. 2022. N3H-core: Neuron-designed neural network accelerator via FPGA-based heterogeneous computing cores. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 112–122.

- [13] Hyeonseok Hong, Dahun Choi, Namjoon Kim, Haein Lee, Beomjin Kang, Huibeom Kang, and Hyun Kim. 2024. Survey of convolutional neural network accelerators on field-programmable gate array platforms: Architectures and optimization techniques. *Journal of Real-Time Image Processing* 21, 3 (2024), 64.
- [14] FPGA Intel. 2017. *Intel® Stratix® 10 Variable Precision DSP Blocks User Guide*.
- [15] Andrey Kuzmin, Markus Nagel, Mart Van Baalen, Arash Behboodi, and Tijmen Blankevoort. 2023. Pruning vs quantization: Which is better? *Advances in Neural Information Processing Systems* 36 (2023), 62414–62427.
- [16] Cecilia Latotzke, Tim Ciesielski, and Tobias Gemmeke. 2019. Design of high-throughput mixed-precision CNN accelerators on FPGA. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 4 (Dec. 2019), 697–711.
- [17] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo. 2018. UNPU: A 50.6TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision. In *Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers*. 218–220.
- [18] Jie Li, Dingjiang Yan, Fangzhou He, Zhicheng Dong, and Mingfei Jiang. 2024. A mixed-precision transformer accelerator with vector tiling systolic array for license plate recognition in unconstrained scenarios. *IEEE Transactions on Intelligent Transportation Systems* (2024).
- [19] Runzhou Li, Bo Jiang, and Hong Xu. 2023. Mixed DSP packing method for convolutional neural network on FPGA. In *Proceedings of the 6th International Conference on Computer Information Science and Application Technology (CISAT 2023)*, Vol. 12800. SPIE, 578–585.
- [20] Xinyu Liu, Tao Wang, Jiaming Yang, Chenwei Tang, and Jiancheng Lv. 2024. MPQ-YOLO: Ultra low mixed-precision quantization of YOLO for edge devices deployment. *Neurocomputing* 574 (2024), 127210.
- [21] Xinyan Liu, Xiao Wu, Haikuo Shao, and Zhongfeng Wang. 2024. A flexible FPGA-based accelerator for efficient inference of multi-precision CNNs. In *Proceedings of the 2024 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [22] Xiaoyin Ma, Walid A Najjar, and Amit K Roy-Chowdhury. 2014. Evaluation and acceleration of high-throughput fixed-point object detection on FPGAs. *IEEE Transactions on Circuits and Systems for Video Technology* 25, 6 (2014), 1051–1062.
- [23] L. Mei, M. Dandekar, D. Rodopoulos, J. Constantin; P. Debacker, and R. Lauwereins 2019. Sub-word parallel precision-scalable MAC engines for efficient embedded DNN inference. In *Proceedings of the IEEE 1st International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. 6–10.
- [24] B. Moons, B. De Brabandere, L. Van Gool, and M. Verhelst. 2016. Energy-efficient ConvNets through approximate computing. In *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*. 1–8.
- [25] B. Moons, R. Uytterhoeven, W. Dehaene, and M. Verhelst. 2017. DVAFS: Trading computational accuracy for energy through dynamic-voltage-accuracy-frequency-scaling. In *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition (DATE)*. 488–493.
- [26] Yaozhong Ou, Wei-Han Yu, Ka-Fai Un, Chi-Hang Chan, and Yan Zhu. 2024. A 119.64 GOPs/W FPGA-Based ResNet50 mixed-precision accelerator using the dynamic DSP packing. *IEEE Transactions on Circuits and Systems II: Express Briefs* (2024).
- [27] Biagio Peccerillo, Mirco Mannino, Andrea Mondelli, and Sandro Bartolini. 2022. A survey on hardware accelerators: Taxonomy, trends, challenges, and perspectives. *Journal of Systems Architecture* 129 (2022), 102561. DOI: <https://doi.org/10.1016/j.sysarc.2022.102561>
- [28] Peng Peng, Mingyu You, Kai Jiang, Youzao Lian, and Weisheng Xu. 2023. MBFQuant: A multiplier-bitwidth-fixed, mixed-precision quantization method for mobile CNN-based applications. *Trans. Img. Proc.* 32 (Jan. 2023), 2438–2453. DOI: <https://doi.org/10.1109/TIP.2023.3268562>
- [29] Jiantao Qiu, Jie Wang, Song Yao, Kaiyuan Guo, Boxun Li, Erjin Zhou, Jincheng Yu, Tianqi Tang, Ningyi Xu, Sen Song, Yu Wang, and Huazhong Yang. 2016. Going deeper with embedded FPGA platform for convolutional neural network. In *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-programMable Gate Arrays*. 26–35.
- [30] S. Sharify, A. D. Lascorz, K. Siu, P. Judd, and A. Moshovos. 2018. Loom: Exploiting weight and activation precisions to accelerate convolutional neural networks. In *Proceedings of the ACM/EDAC/IEEE Design Automation Conference (DAC)*. 20:1–20:6.
- [31] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, J. K. Kim, V. Chandra, and H. Esmaeilzadeh. 2018. Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural networks. In *Proceedings of the 45th IEEE International Symposium on Computer Architecture (ISCA)*. 764–775.
- [32] Hardik Sharma, Jongse Park, Naveen Suda, Liangzhen Lai, Benson Chau, Joon Kyung Kim, Vikas Chandra, and Hadi Esmaeilzadeh. 2018. Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network. In *Proceedings of the 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 764–775.
- [33] D. Shin, J. Lee, J. Lee, and H.-J. Yoo. 2017. DNPU: An 8.1TOPS/W reconfigurable CNN-RNN processor for general-purpose deep neural networks. In *Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers*. 240–241.

- [34] Jan Sommer, M Akif Özkan, Oliver Keszocze, and Jürgen Teich. 2022. Dsp-packing: Squeezing low-precision arithmetic into fpga dsp blocks. In *Proceedings of the 2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 160–166.
- [35] Mingcong Song, Jiaqi Zhang, Huixiang Chen, and Tao Li. 2018. Towards efficient microarchitectural design for accelerating unsupervised gan-based deep learning. In *Proceedings of the 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 66–77.
- [36] Mengshu Sun, Zhengang Li, Alec Lu, Yanyu Li, Sung-En Chang, Xiaolong Ma, Xue Lin, and Zhenman Fang. 2022. Film-qnn: Efficient fpga acceleration of deep neural networks with intra-layer, mixed-precision quantization. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 134–145.
- [37] JianRong Wang, Zhijun He, Hongbo Zhao, and Rongke Liu. 2024. Low-bit mixed-precision quantization and acceleration of CNN for FPGA deployment. *IEEE Transactions on Emerging Topics in Computational Intelligence* (2024).
- [38] Kuan Wang, Zhijian Liu, Yujun Lin, Ji Lin, and Song Han. 2019. Haq: Hardware-aware automated quantization with mixed precision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8612–8620.
- [39] Chen Wu, Jinming Zhuang, Kun Wang, and Lei He. 2021. MP-OPU: A mixed precision FPGA-based overlay processor for convolutional neural networks. In *Proceedings of the 2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 33–37.
- [40] Jiajun Wu, Jiajun Zhou, Yizhao Gao, Yuhao Ding, Ngai Wong, and Hayden Kwok-Hay So. 2023. Msd: Mixing signed digit representations for hardware-efficient dnn acceleration on fpga with heterogeneous resources. In *Proceedings of the 2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 94–104.
- [41] Runhua Zhang, Hongxu Jiang, Wei Wang, and Jinhao Liu. 2025. Optimization methods, challenges, and opportunities for edge inference: A comprehensive survey. *Electronics* 14, 7 (2025), 1345.
- [42] Xiaotian Zhao, Ruge Xu, Yimin Gao, Vaibhav Verma, Mircea R Stan, and Xinfei Guo. 2024. Edge-MPQ: Layer-wise mixed-precision quantization with tightly integrated versatile inference units for edge computing. *IEEE Transactions on Computers* (2024).

Received 22 July 2025; revised 22 July 2025; accepted 29 July 2025